

ATNX — Claude Code Build Instructions

This document covers the full weekend build in **four sequential, independently-shippable chunks**. Run each chunk as a separate Claude Code session. After each chunk, verify the app still works end-to-end before moving on. If anything breaks, you'll know exactly which chunk regressed.

Prerequisites (should all be done):

- Supabase project set up with schema, RLS, extensions, storage bucket (see `ATNX_Supabase_Setup_Instructions.md`)
- Google OAuth configured in both Google Cloud and Supabase
- `.env.local` has `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`
- App is on Next.js App Router

The four chunks:

1. **Persistence Layer** — replace in-memory `lib/store.ts` with Supabase, captures persist across restarts
2. **Google Auth Integration** — sign-in gate on `/app/*`, user profile bootstrap, handle editing
3. **Trade Flow on Supabase** — sim balance and positions persist, PnL calculated server-side
4. **Taxonomy MVP + Admin Dashboard** — trigram entity matching on capture, admin page at `/admin`

Each section below is designed as a single-shot Claude Code prompt. Paste the whole section into Claude Code when you're ready to ship that chunk.

CHUNK 1 — Persistence Layer

Context for Claude Code

You are working on ATNX, a Next.js App Router app in `atnx-web/`. The app currently uses an in-memory store in `lib/store.ts` for captures, and a React Context (`DemoContext`) for sim trading state. Both reset on server restart and don't persist across devices.

Your job in this chunk: migrate all data persistence to Supabase. Auth is NOT yet integrated in this chunk — treat captures as anonymous for now (we'll add `user_id` in chunk 2). Just get the data flowing through Postgres.

Environment

`.env.local` has:

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
```

Supabase already has these tables: `user_profiles`, `markets`, `captures`, `vi_history`, `sim_balances`, `positions`, `moderation_log`. RLS is enabled on all. For this chunk, server-side writes use the `service_role` key to bypass RLS (since there's no auth yet).

Step-by-step tasks

1.1 — Install Supabase libraries

```
bash

npm install @supabase/supabase-js @supabase/ssr
```

1.2 — Create Supabase clients

Create `lib/supabase/client.ts`:

```
typescript

import { createBrowserClient } from '@supabase/ssr';

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}
```

Create `lib/supabase/server.ts`:

```
typescript
```

```

import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Server Component - can't set cookies; fine, middleware will handle
          }
        },
      },
    },
  );
}

```

Create `lib/supabase/admin.ts` (for service_role operations — bypasses RLS, NEVER import from client components):

```

typescript

import { createClient } from '@supabase/supabase-js';

export function createAdminClient() {
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,
    { auth: { persistSession: false } }
  );
}

```

1.3 — TypeScript types for schema

Create `lib/supabase/types.ts` with types matching the Supabase schema:

```
typescript
```

```
export type Market = {
  id: string;
  canonical_entity_id: string | null;
  entity_name: string;
  entity_name_normalized: string;
  entity_type: string | null;
  thumbnail_url: string | null;
  current_vi: number;
  vi_last_updated: string | null;
  phase: number;
  is_graduated: boolean;
  total_captures: number;
  total_volume_usd: number;
  network: 'simulated' | 'devnet' | 'mainnet';
  on_chain_pda: string | null;
  trading_mode: 'sim' | 'live';
  deleted_at: string | null;
  created_at: string;
};
```

```
export type Capture = {
  id: string;
  user_id: string | null;
  market_id: string | null;
  image_url: string | null;
  ocr_text: string | null;
  source_url: string | null;
  raw_ai_response: Record<string, unknown> | null;
  embedding_text: number[] | null;
  embedding_image: number[] | null;
  perceptual_hash: number | null;
  confidence_score: number | null;
  resolution_status: 'pending' | 'resolved' | 'review' | 'new_entity';
  deleted_at: string | null;
  created_at: string;
};
```

```
export type Position = {
  id: string;
  user_id: string;
  market_id: string;
  direction: 'long' | 'short';
  size_usd: number;
```

```

    entry_vi: number;
    entry_price: number;
    leverage: number;
    network: 'simulated' | 'devnet' | 'mainnet';
    tx_signature: string | null;
    opened_at: string;
    closed_at: string | null;
    exit_vi: number | null;
    exit_price: number | null;
    realized_pnl: number | null;
    status: 'open' | 'closed';
  };

```

```

export type SimBalance = {
  user_id: string;
  balance_usd: number;
  total_pnl_realized: number;
  total_trades: number;
  updated_at: string;
};

```

```

export type UserProfile = {
  id: string;
  handle: string;
  email: string | null;
  avatar_url: string | null;
  wallet_address: string | null;
  auth_methods: string[];
  role: 'user' | 'admin' | 'moderator';
  created_at: string;
  updated_at: string;
};

```

```

export type ViHistoryPoint = {
  id: number;
  market_id: string;
  vi: number;
  recorded_at: string;
};

```

1.4 — Replace `lib/store.ts`

Read the existing `lib/store.ts` to understand what functions it exports (likely things like `addCapture`, `getCaptures`, `getCaptureById`). Replace each function with the Supabase

equivalent.

Important: since we don't have auth yet, temporarily treat all captures as anonymous by storing them with `user_id = null`. RLS will block this via the anon key, so **use the admin client (service_role)** for writes in this chunk. This is a temporary workaround — chunk 2 will add proper `user_id` attribution.

Example structure for the new `lib/store.ts`:

```
typescript
```

```

import { createAdminClient } from '@lib/supabase/admin';
import type { Capture, Market } from '@lib/supabase/types';

const supabase = createAdminClient();

export async function addCapture(input: {
  image_url: string;
  ocr_text?: string;
  source_url?: string;
  raw_ai_response: Record<string, unknown>;
  entity_name: string;
  entity_type: string;
}): Promise<Capture> {
  // 1. Find or create market (weekend MVP: exact name match; chunk 4 adds trigram m
  const normalized = input.entity_name.toLowerCase().trim();

  let { data: market } = await supabase
    .from('markets')
    .select('*')
    .eq('entity_name_normalized', normalized)
    .maybeSingle();

  if (!market) {
    const { data: newMarket, error } = await supabase
      .from('markets')
      .insert({
        entity_name: input.entity_name,
        entity_name_normalized: normalized,
        entity_type: input.entity_type,
      })
      .select()
      .single();
    if (error) throw error;
    market = newMarket;
  }

  // 2. Create capture
  const { data: capture, error } = await supabase
    .from('captures')
    .insert({
      market_id: market.id,
      image_url: input.image_url,
      ocr_text: input.ocr_text,

```

```

        source_url: input.source_url,
        raw_ai_response: input.raw_ai_response,
        resolution_status: 'resolved',
    })
    .select()
    .single();
    if (error) throw error;

    // 3. Increment market capture count
    await supabase
        .from('markets')
        .update({ total_captures: (market.total_captures ?? 0) + 1 })
        .eq('id', market.id);

    return capture;
}

export async function getCaptures(limit = 50): Promise<(Capture & { market: Market }) {
    const { data, error } = await supabase
        .from('captures')
        .select('*', market:markets('*'))
        .is('deleted_at', null)
        .order('created_at', { ascending: false })
        .limit(limit);
    if (error) throw error;
    return data ?? [];
}

export async function getCaptureById(id: string) {
    const { data, error } = await supabase
        .from('captures')
        .select('*', market:markets('*'))
        .eq('id', id)
        .maybeSingle();
    if (error) throw error;
    return data;
}

```

Adapt the exported function signatures to match whatever the existing `lib/store.ts` exposes so existing callers don't break.

1.5 — Capture image uploads to Supabase Storage

Wherever the app currently handles capture images (likely in the `/api/captures` POST route),

upload the image to Supabase Storage `captures` bucket instead of storing base64 or local files.

Since there's no auth yet, upload to a `anonymous/` prefix:

typescript

```
const fileName = `anonymous/${crypto.randomUUID()}.png`;
const { data, error } = await supabase.storage
  .from('captures')
  .upload(fileName, imageBuffer, { contentType: 'image/png' });

if (error) throw error;

const { data: { publicUrl } } = supabase.storage
  .from('captures')
  .getPublicUrl(fileName);

// Use publicUrl as the capture's image_url
```

Note: the `anonymous/` prefix violates the RLS policy we wrote (which requires the first folder to match `auth.uid()`). That's why this chunk uses the admin client for Storage writes too. Chunk 2 will switch to per-user folders.

1.6 — VI history persistence

Find where the Google Trends / virality score updates happen (likely in `lib/trends.ts` or `lib/trends-cache.ts`). Every time a VI score is computed for a market, also insert a row into `vi_history`:

typescript

```
await supabase
  .from('vi_history')
  .insert({ market_id: marketId, vi: newScore });

await supabase
  .from('markets')
  .update({
    current_vi: newScore,
    vi_last_updated: new Date().toISOString(),
  })
  .eq('id', marketId);
```

For the dashboard sparklines, read recent history:

typescript

```
export async function getViHistory(marketId: string, points = 50) {
  const { data, error } = await supabase
    .from('vi_history')
    .select('vi, recorded_at')
    .eq('market_id', marketId)
    .order('recorded_at', { ascending: false })
    .limit(points);
  if (error) throw error;
  return (data ?? []).reverse(); // chronological order for charts
}
```

1.7 — Verify chunk 1 works end-to-end

1. Start dev server (`npm run dev`)
2. Open the extension, capture a meme
3. Confirm the capture appears on the dashboard
4. Restart the dev server
5. Refresh the dashboard — the capture should still be there
6. Check Supabase Table Editor — `markets` has a row, `captures` has a row pointing to that market
7. Check Supabase Storage — `captures` bucket has the image file
8. Make another capture of the same meme — confirm it joins the existing market (same `market_id`) rather than creating a duplicate

Chunk 1 is done when captures persist across server restarts and same-name captures merge to the same market.

CHUNK 2 — Google Auth Integration

Context for Claude Code

Chunk 1 is shipped — captures persist in Supabase but are all anonymous. Now add Google sign-in so every action is tied to a real user. After this chunk, the app requires auth for `/app/*` routes, and `user_id` is correctly populated on captures and positions.

Step-by-step tasks

2.1 — Auth middleware

Create `middleware.ts` in the repo root (or `atnx-web/` root — wherever `next.config.js` lives):

typescript

```

import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';

export async function middleware(request: NextRequest) {
  let response = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value)
          );
          response = NextResponse.next({ request });
          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options)
          );
        },
      },
    },
  );

  // IMPORTANT: do NOT put any code between createServerClient and getUser
  const { data: { user } } = await supabase.auth.getUser();

  const path = request.nextUrl.pathname;
  const isProtected = path.startsWith('/app') || path.startsWith('/admin');
  const isAuthCallback = path.startsWith('/auth/callback');

  if (isProtected && !user && !isAuthCallback) {
    const url = request.nextUrl.clone();
    url.pathname = '/';
    url.searchParams.set('redirect', path);
    return NextResponse.redirect(url);
  }

  return response;
}

```

```
export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).
  ],
};
```

2.2 — Sign-in button on landing page

On the landing page (`app/page.tsx`) or wherever the `/` route is defined), the existing `Launch App →` button should trigger Google sign-in. Update the button to be a client component that calls:

```
typescript

'use client';
import { createClient } from '@lib/supabase/client';

export function LaunchAppButton() {
  const supabase = createClient();

  async function handleSignIn() {
    const { error } = await supabase.auth.signInWithOAuth({
      provider: 'google',
      options: {
        redirectTo: `${window.location.origin}/auth/callback`,
      },
    });
    if (error) console.error('Sign-in error:', error);
  }

  return (
    <button onClick={handleSignIn} className="...existing CMYK styling">
      Launch App →
    </button>
  );
}
```

If the user is already signed in, the button should route directly to `/app` without going through OAuth. Check auth state in a parent server component and conditionally render.

2.3 — OAuth callback route

Create `app/auth/callback/route.ts`:

typescript

```
import { createClient } from '@lib/supabase/server';
import { NextResponse } from 'next/server';

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get('code');
  const redirect = searchParams.get('redirect') ?? '/app';

  if (code) {
    const supabase = await createClient();
    const { error } = await supabase.auth.exchangeCodeForSession(code);
    if (!error) {
      return NextResponse.redirect(`${origin}${redirect}`);
    }
  }

  return NextResponse.redirect(`${origin}/?error=auth_failed`);
}
```

2.4 — Attribute captures to the signed-in user

Update the `/api/captures` POST route to read the authenticated user from the server client and attribute the capture to them:

typescript

```

import { createClient } from '@lib/supabase/server';

export async function POST(request: Request) {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();

  if (!user) {
    return new Response('Unauthorized', { status: 401 });
  }

  // ... existing AI analysis logic ...

  // Upload image to user's own folder (matches RLS policy)
  const fileName = `${user.id}/${crypto.randomUUID()}.png`;
  const { error: uploadError } = await supabase.storage
    .from('captures')
    .upload(fileName, imageBuffer, { contentType: 'image/png' });
  if (uploadError) throw uploadError;

  const { data: { publicUrl } } = supabase.storage
    .from('captures')
    .getPublicUrl(fileName);

  // Use user's session client for capture insert (RLS enforces user_id = auth.uid())
  const { data: capture, error } = await supabase
    .from('captures')
    .insert({
      user_id: user.id,
      market_id: market.id,
      image_url: publicUrl,
      // ... other fields
    })
    .select()
    .single();

  if (error) throw error;
  return Response.json(capture);
}

```

Update `lib/store.ts` — replace the admin client usage with the server session client for user-scoped operations. Keep the admin client only for operations that need to bypass RLS (e.g., creating markets, writing VI history).

2.5 — Handle editing UI

Create `app/app/settings/page.tsx` (or wherever settings lives) with a form to edit the user's handle. Default value from `user_profiles.handle`.

Validation rules (enforce on submit):

- 3-24 characters
- Alphanumeric + `_` + `#` only
- Blocklist (case-insensitive): `admin`, `atnx`, `official`, `system`, `mod`, `moderator`, `support`. Also reject any handle starting with `Attn_seeker#` to prevent impersonating the default pattern (unless it's their current auto-assigned one).
- Uniqueness handled by the DB (unique constraint on `handle`) — catch the error and surface as "That handle is taken".

typescript

```
'use client';
// ... form UI ...

async function updateHandle(newHandle: string) {
  const supabase = createClient();
  const { error } = await supabase
    .from('user_profiles')
    .update({ handle: newHandle, updated_at: new Date().toISOString() })
    .eq('id', user.id);

  if (error?.code === '23505') {
    toast('That handle is taken');
  } else if (error) {
    toast('Update failed');
  } else {
    toast('Handle updated');
  }
}
```

2.6 — Show user handle in nav

Wherever the top nav is defined, fetch the current user's handle from `user_profiles` and display it (e.g., "Attn_seeker#4821" with a dropdown for Settings / Sign out). Keep it simple.

2.7 — Verify chunk 2 works end-to-end

1. Sign out if signed in. Open `/` — landing page shows.
2. Click Launch App — redirects to Google OAuth.
3. Sign in. Redirected back to `/app`.
4. Check Supabase → Authentication → Users: your email appears.
5. Check `user_profiles` table: row exists with random `Attn_seeker#XXXX` handle.
6. Check `sim_balances` table: row exists with \$10000 balance.
7. Open the extension, capture a meme.
8. Check `captures` table: row has your `user_id` populated.
9. Check Storage: image uploaded under `{your_user_id}/...png`.
10. Navigate to `/app/settings`, change handle to "TestHandle_123", save, reload — handle persists.
11. Try setting handle to "admin" — should be rejected.
12. Try to visit `/app` in an incognito window — redirected to `/`.

Chunk 2 is done when only authenticated users can use the app and every action is correctly attributed.

2.8 — Promote yourself to admin

After first successful sign-in, go to Supabase SQL Editor and run:

```
sql

update public.user_profiles
set role = 'admin'
where email = 'your.email@gmail.com';

select id, handle, email, role from public.user_profiles where role = 'admin';
```

Sign out and back in so your JWT picks up the new role.

CHUNK 3 — Trade Flow on Supabase

Context for Claude Code

Chunks 1 and 2 are shipped. Captures persist, users sign in with Google, markets exist. Now replace the in-memory sim trading (`DemoContext`) with real Supabase-backed positions and

balance tracking.

Step-by-step tasks

3.1 — Read current DemoContext state

Before writing code, read `contexts/DemoContext.tsx` (or wherever it lives) and understand the current shape:

- What state does it hold? (positions, balance, PnL history, live ticker state)
- What actions does it expose? (openPosition, closePosition, etc.)
- Which components consume it?

You'll replace the data sources one function at a time, keeping the same interface where possible so downstream components don't need rewriting.

3.2 — Server actions for trade operations

Create `app/app/actions/trading.ts` using Next.js Server Actions (so writes happen server-side with the user's session cookie, and RLS enforces correct user scoping):

```
typescript
```

```

'use server';

import { createClient } from '@lib/supabase/server';
import { revalidatePath } from 'next/cache';

export async function openPosition(input: {
  market_id: string;
  direction: 'long' | 'short';
  size_usd: number;
  leverage?: number;
}) {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error('Unauthorized');

  // Get current market state for entry VI and price
  const { data: market } = await supabase
    .from('markets')
    .select('id, current_vi')
    .eq('id', input.market_id)
    .single();
  if (!market) throw new Error('Market not found');

  // Check user balance
  const { data: balance } = await supabase
    .from('sim_balances')
    .select('balance_usd')
    .eq('user_id', user.id)
    .single();
  if (!balance || balance.balance_usd < input.size_usd) {
    throw new Error('Insufficient balance');
  }

  // Entry price derivation: for the weekend MVP, use VI directly as price
  // (full PMM pricing comes later). price = current_vi normalized to a USD number.
  const entry_price = market.current_vi;

  // Insert position + debit balance atomically via Postgres function
  // (or sequential ops – the minor race risk is fine for sim)
  const { data: position, error } = await supabase
    .from('positions')
    .insert({
      user_id: user.id,

```

```

    market_id: input.market_id,
    direction: input.direction,
    size_usd: input.size_usd,
    entry_vi: market.current_vi,
    entry_price,
    leverage: input.leverage ?? 1,
    network: 'simulated',
    status: 'open',
  })
  .select()
  .single();
if (error) throw error;

await supabase
  .from('sim_balances')
  .update({
    balance_usd: balance.balance_usd - input.size_usd,
    total_trades: (await getTradeCount(user.id)) + 1,
    updated_at: new Date().toISOString(),
  })
  .eq('user_id', user.id);

revalidatePath('/app');
revalidatePath('/app/portfolio');
return position;
}

export async function closePosition(positionId: string) {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error('Unauthorized');

  const { data: position } = await supabase
    .from('positions')
    .select('*', market:markets(current_vi)')
    .eq('id', positionId)
    .eq('user_id', user.id)
    .eq('status', 'open')
    .single();
  if (!position) throw new Error('Position not found');

  const exit_price = position.market.current_vi;
  const exit_vi = position.market.current_vi;

```

```

// PnL calc: simple linear for weekend MVP
//   long:  pnl = size * (exit / entry - 1) * leverage
//   short: pnl = size * (1 - exit / entry) * leverage
const priceRatio = exit_price / position.entry_price;
const directionMultiplier = position.direction === 'long' ? (priceRatio - 1) : (1
const realized_pnl = position.size_usd * directionMultiplier * position.leverage;

await supabase
  .from('positions')
  .update({
    status: 'closed',
    closed_at: new Date().toISOString(),
    exit_vi,
    exit_price,
    realized_pnl,
  })
  .eq('id', positionId);

// Return margin + PnL to balance
const { data: balance } = await supabase
  .from('sim_balances')
  .select('balance_usd, total_pnl_realized')
  .eq('user_id', user.id)
  .single();

await supabase
  .from('sim_balances')
  .update({
    balance_usd: balance!.balance_usd + position.size_usd + realized_pnl,
    total_pnl_realized: balance!.total_pnl_realized + realized_pnl,
    updated_at: new Date().toISOString(),
  })
  .eq('user_id', user.id);

revalidatePath('/app');
revalidatePath('/app/portfolio');
return { ...position, realized_pnl };
}

async function getTradeCount(userId: string): Promise<number> {
  const supabase = await createClient();
  const { data } = await supabase
    .from('sim_balances')
    .select('total_trades')

```

```
    .eq('user_id', userId)
    .single();
    return data?.total_trades ?? 0;
  }
```

3.3 — Update DemoContext to read from Supabase

Refactor `DemoContext` (or whatever holds trading state):

- Remove the local `balance`, `positions` state
- On mount, fetch balance + open positions from Supabase for the current user
- Replace `openPosition()` / `closePosition()` with calls to the server actions
- Keep the `useFakeTicker` simulated price movement for dashboard chart animation (that's UI sugar, doesn't need persistence)

Structure:

typescript

```

'use client';

import { createContext, useContext, useEffect, useState } from 'react';
import { createClient } from '@lib/supabase/client';
import { openPosition as serverOpenPosition, closePosition as serverClosePosition }

type DemoContextType = {
  balance: number;
  positions: Position[];
  openPosition: (input: OpenPositionInput) => Promise<void>;
  closePosition: (id: string) => Promise<void>;
  refresh: () => Promise<void>;
};

const DemoContext = createContext<DemoContextType | null>(null);

export function DemoProvider({ children }: { children: React.ReactNode }) {
  const [balance, setBalance] = useState(10000);
  const [positions, setPositions] = useState<Position[]>([]);
  const supabase = createClient();

  async function refresh() {
    const { data: { user } } = await supabase.auth.getUser();
    if (!user) return;

    const [{ data: bal }, { data: pos }] = await Promise.all([
      supabase.from('sim_balances').select('balance_usd').eq('user_id', user.id).single(),
      supabase.from('positions').select('*', market:markets('*')).eq('user_id', user.id).in(
    ]);

    if (bal) setBalance(bal.balance_usd);
    if (pos) setPositions(pos);
  }

  useEffect(() => { refresh(); }, []);

  async function openPosition(input: OpenPositionInput) {
    await serverOpenPosition(input);
    await refresh();
  }

  async function closePosition(id: string) {
    await serverClosePosition(id);
  }

```

```

    await refresh();
  }

  return (
    <DemoContext.Provider value={{ balance, positions, openPosition, closePosition,
      {children}
    </DemoContext.Provider>
  );
}

export function useDemoContext() {
  const ctx = useContext(DemoContext);
  if (!ctx) throw new Error('useDemoContext must be used within DemoProvider');
  return ctx;
}

```

3.4 — Portfolio page uses real data

Update `app/app/portfolio/page.tsx` to fetch both open and closed positions from Supabase for the current user. Show open positions with live PnL computed client-side using the market's current VI. Show closed positions with realized PnL from the database.

Open position live PnL formula (client-side display):

typescript

```

const livePnl = position.direction === 'long'
  ? position.size_usd * (currentVi / position.entry_vi - 1) * position.leverage
  : position.size_usd * (1 - currentVi / position.entry_vi) * position.leverage;

```

3.5 — Verify chunk 3 works end-to-end

1. Sign in, confirm balance shows \$10,000
2. Open a \$500 long position on any market — balance drops to \$9,500, position visible in portfolio
3. Restart dev server, reload — position still there with correct data
4. Sign out, sign in on a different browser (same Google account) — position and balance sync
5. Close the position — realized PnL computed, balance adjusts
6. Check `positions` table — status is `closed`, `realized_pnl` populated
7. Try opening a \$20,000 position — rejected with "Insufficient balance"
8. Try to access `/app/portfolio` while signed out — middleware redirects to `/`

Chunk 3 is done when sim trading is fully persistent and multi-device.

CHUNK 4 — Taxonomy MVP + Admin Dashboard

Context for Claude Code

Chunks 1-3 are shipped. The app has persistent captures, auth, and sim trading. Now add two things: (a) smarter entity resolution so near-duplicate captures merge to the same market, and (b) an admin dashboard for moderation.

Step-by-step tasks

4.1 — Trigram entity matching on capture ingestion

In the capture ingestion pipeline, replace the exact-name match from chunk 1 with fuzzy matching using the `pg_trgm` trigram similarity index.

In `lib/store.ts` (or wherever market resolution happens), update the logic:

```
typescript
```

```

async function resolveOrCreateMarket(
  entity_name: string,
  entity_type: string
): Promise<{ id: string; is_new: boolean }> {
  const normalized = entity_name.toLowerCase().trim();
  const adminSupabase = createAdminClient();

  // Fuzzy match against existing non-deleted markets
  const { data: matches, error } = await adminSupabase.rpc('find_similar_market', {
    query_name: normalized,
    threshold: 0.85,
  });
  if (error) throw error;

  if (matches && matches.length > 0) {
    return { id: matches[0].id, is_new: false };
  }

  // No match – create new market
  const { data: newMarket, error: insertError } = await adminSupabase
    .from('markets')
    .insert({
      entity_name,
      entity_name_normalized: normalized,
      entity_type,
    })
    .select('id')
    .single();
  if (insertError) throw insertError;

  return { id: newMarket.id, is_new: true };
}

```

The RPC function needs to exist on the database. Add this via the Supabase SQL Editor (one-time setup, outside Claude Code's work):

```
sql
```

```

create or replace function public.find_similar_market(
  query_name text,
  threshold real default 0.85
)
returns table (id uuid, entity_name text, similarity real)
language sql
stable
as $$
  select
    m.id,
    m.entity_name,
    similarity(m.entity_name_normalized, query_name) as similarity
  from public.markets m
  where
    m.deleted_at is null
    and similarity(m.entity_name_normalized, query_name) > threshold
  order by similarity desc
  limit 1;
$$;

```

Claude Code: remind the user to run the above SQL in Supabase before testing this chunk.

4.2 — Confidence score recording

When a capture is created, store the similarity score in `captures.confidence_score` so admins can later review low-confidence matches:

```

typescript

// After resolveOrCreateMarket finds a match with similarity 0.87
await supabase
  .from('captures')
  .update({
    confidence_score: similarity,
    resolution_status: similarity > 0.95 ? 'resolved' : 'review',
  })
  .eq('id', captureId);

```

If similarity > 0.95: auto-resolved. If 0.85-0.95: marked for review. If below 0.85: new market created, status stays 'resolved' (new entity was the right call).

4.3 — Admin dashboard route

Create `app/admin/page.tsx` (server component) that checks admin status on the server and

renders the dashboard:

typescript

```
import { createClient } from '@lib/supabase/server';
import { redirect } from 'next/navigation';
import { AdminDashboard } from './dashboard-client';

export default async function AdminPage() {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();

  if (!user) redirect('/');

  const { data: profile } = await supabase
    .from('user_profiles')
    .select('role')
    .eq('id', user.id)
    .single();

  if (!profile || !['admin', 'moderator'].includes(profile.role)) {
    redirect('/app');
  }

  return <AdminDashboard />;
}
```

4.4 — Admin dashboard UI

Create `app/admin/dashboard-client.tsx` (client component) with three tabs:

1. **Markets** — table of all markets (including soft-deleted). Columns: `entity_name`, `total_captures`, `current_vi`, `network`, `status` (active/deleted), `created_at`. Row actions: Edit name, Soft-delete, Restore (if deleted).
2. **Captures** — table of all captures with low confidence (`resolution_status = 'review'` or `confidence_score < 0.95`). Columns: `thumbnail`, `user handle`, `entity_name` (from joined market), `confidence_score`, `created_at`. Row actions: Approve, Reassign to different market, Soft-delete.
3. **Moderation Log** — append-only list of all admin actions, newest first. Columns: `admin handle`, `action`, `target_type`, `target_id`, `reason`, `created_at`.

Use the CMYK styling consistent with the rest of ATNX (black bg, cyan/magenta/yellow accents, JetBrains Mono / monospace).

4.5 — Admin server actions

Create `app/admin/actions.ts` for all admin mutations. Every admin write MUST also insert a row into `moderation_log` in the same transaction. The cleanest way is a Postgres function:

Add to Supabase SQL Editor:

```
sql
```

```
create or replace function public.admin_soft_delete_market(
    market_id uuid,
    reason text default null
)
returns void
language plpgsql
security definer
as $$
declare
    acting_user uuid := auth.uid();
begin
    if not public.is_admin() then
        raise exception 'Not authorized';
    end if;

    update public.markets
        set deleted_at = now()
        where id = market_id;

    insert into public.moderation_log (admin_user_id, action, target_type, target_id,
        values (acting_user, 'soft_delete_market', 'market', market_id, reason);
end;
$$;

create or replace function public.admin_restore_market(
    market_id uuid,
    reason text default null
)
returns void
language plpgsql
security definer
as $$
declare
    acting_user uuid := auth.uid();
begin
    if not public.is_admin() then
        raise exception 'Not authorized';
    end if;

    update public.markets
        set deleted_at = null
        where id = market_id;
```

```

insert into public.moderation_log (admin_user_id, action, target_type, target_id,
    values (acting_user, 'restore_market', 'market', market_id, reason);
end;
$$;

create or replace function public.admin_edit_market_name(
    market_id uuid,
    new_name text,
    reason text default null
)
returns void
language plpgsql
security definer
as $$
declare
    acting_user uuid := auth.uid();
    old_name text;
begin
    if not public.is_admin() then
        raise exception 'Not authorized';
    end if;

    select entity_name into old_name from public.markets where id = market_id;

    update public.markets
        set entity_name = new_name,
            entity_name_normalized = lower(trim(new_name))
        where id = market_id;

    insert into public.moderation_log (admin_user_id, action, target_type, target_id,
        values (acting_user, 'edit_market_name', 'market', market_id, reason,
            jsonb_build_object('old_name', old_name, 'new_name', new_name));
end;
$$;

create or replace function public.admin_soft_delete_capture(
    capture_id uuid,
    reason text default null
)
returns void
language plpgsql
security definer
as $$
declare

```

```

    acting_user uuid := auth.uid();
begin
    if not public.is_admin() then
        raise exception 'Not authorized';
    end if;

    update public.captures
        set deleted_at = now()
        where id = capture_id;

    insert into public.moderation_log (admin_user_id, action, target_type, target_id,
        values (acting_user, 'soft_delete_capture', 'capture', capture_id, reason);
end;
$$;

create or replace function public.admin_reassign_capture(
    capture_id uuid,
    new_market_id uuid,
    reason text default null
)
returns void
language plpgsql
security definer
as $$
declare
    acting_user uuid := auth.uid();
    old_market_id uuid;
begin
    if not public.is_admin() then
        raise exception 'Not authorized';
    end if;

    select market_id into old_market_id from public.captures where id = capture_id;

    update public.captures
        set market_id = new_market_id,
            resolution_status = 'resolved'
        where id = capture_id;

    insert into public.moderation_log (admin_user_id, action, target_type, target_id,
        values (acting_user, 'reassign_capture', 'capture', capture_id, reason,
            jsonb_build_object('old_market_id', old_market_id, 'new_market_id', new_

```

```
end;  
$$;
```

Claude Code: remind the user to run the above SQL in Supabase before testing this chunk.

Then the server actions in `app/admin/actions.ts` just call these functions:

```
typescript  
  
'use server';  
  
import { createClient } from '@lib/supabase/server';  
import { revalidatePath } from 'next/cache';  
  
export async function softDeleteMarket(marketId: string, reason: string) {  
  const supabase = await createClient();  
  const { error } = await supabase.rpc('admin_soft_delete_market', {  
    market_id: marketId,  
    reason,  
  });  
  if (error) throw error;  
  revalidatePath('/admin');  
}  
  
// ... analogous wrappers for restoreMarket, editMarketName, softDeleteCapture, reas
```

4.6 — Link to admin page from user nav

If the logged-in user has `role` of `admin` or `moderator`, show an "Admin" link in the nav dropdown. Check role in the server component that renders the nav.

4.7 — Verify chunk 4 works end-to-end

1. Sign in as your (admin) user
2. Capture the same meme 3 times with slightly different names: "Quantum Cats", "quantum cats", "quantumcats"
 - First creates a new market
 - Second and third should merge to the same market (check `market_id` on captures)
3. Visit `/admin` — dashboard loads
4. Sign out, sign in as a regular (non-admin) user — `/admin` redirects to `/app`

5. Back as admin, soft-delete a market — it disappears from the main dashboard but still shows in admin Markets tab with "deleted" status
6. Restore it — reappears on main dashboard
7. Edit a market's name — change propagates, old name logged in `moderation_log`
8. Check Supabase → Table Editor → `moderation_log` — every admin action has a row

Chunk 4 is done when entity matching correctly merges near-duplicates and admins can moderate with full audit trail.

Post-chunk work (not weekend scope)

After these four chunks ship, the app is feature-complete for the demo-ready hackathon MVP. What's left for the remaining weeks before Colosseum deadline:

- **Anchor program on devnet** — your lane, separate build
- **Full taxonomy pipeline** — SBERT embeddings, HDBSCAN, Wikidata entity linking (other dev's lane)
- **Mobile capture via PWA share target** — other dev's lane
- **Wallet connect (Phantom)** — your lane, after Anchor
- **Demo video + pitch polish** — final week

Everything from this document is forward-compatible with all of those. No refactors needed.

Ordering reminders for Claude Code

Before running chunk 1: Supabase setup complete, env vars in `.env.local`.

Between chunk 4's Claude Code work and testing: Run the RPC SQL (`find_similar_market` + `admin_*` functions) in the Supabase SQL Editor manually. Claude Code should NOT try to create these via the Supabase client — it won't work through the API. They must be created via the SQL Editor or Supabase CLI.

If any chunk breaks something visibly in the app: roll back via git, diagnose, retry. Don't let a broken chunk block the next one.

Ship it.